

Block 2: Von der Anforderung zum Plan

Workshop: Agentic Engineering

Thomas Hein — Dataciders

Wie funktioniert ein Coding Agent?

LLM + Tools + Kontext = Agent

Der Kreislauf:

1. **Prompt** empfangen
2. **Denken** (Reasoning)
3. **Tool aufrufen** (Dateien lesen, Code schreiben, Tests ausführen)
4. **Ergebnis** auswerten

Autocomplete → Chat → Agent

Modus	Wie es funktioniert	Beispiel
Autocomplete	Inline-Vorschlaege beim Tippen	Tab-Completion in IDE
Chat	Frage-Antwort im Seitenpanel	"Erklaere diese Funktion"
Agent	Autonome Ausfuehrung mit Tool-Zugriff	"Implementiere Feature X mit Tests"

Das Oekosystem eines Coding Agents

- **Skills/Rules:** Vordefinierte Workflows und Regeln (z.B. TDD-Skill)
- **Plugins:** Erweiterungen (z.B. Context7 fuer aktuelle Doku)
- **MCP-Server:** Externe Datenquellen anbinden (DB, Jira, Git)
- **Hooks:** Automatische Aktionen bei bestimmten Events

[BILD: MCP Host-Client-Server Architektur — Quelle:

<https://modelcontextprotocol.io/docs/concepts/architecture>]

Quellen & Weiterlesen — Funktionsweise Coding Agent

- [DEV Community: "Forget the Hype: Agents are Loops"](#) — Praxis-Erklärung des Agent-Loop-Konzepts
- [Oracle: "What Is the AI Agent Loop?"](#) — Perceive/Reason/Plan/Act/Observe Schleife
- [Model Context Protocol — Architektur](#) — MCP Host-Client-Server Konzept
- [GitHub Blog: Copilot Ask, Edit, Agent modes](#) — Drei Modi im Vergleich

Kontextmanagement — Das Fundament

"Ein Agent ist nur so gut wie sein Kontext"

- **AGENTS.md / Copilot Instructions** — dem Agenten das Projekt erklären
- **Kontext-Hygiene:** Irrelevantes entfernen, Relevantes fokussieren
- **Token-Budgets:** Nicht alles reipacken — Relevanz > Vollständigkeit

Die 3 Kontext-Schichten

Schicht	Inhalt	Beispiel
Immer aktiv	Konventionen, Architektur, Plattform	AGENTS.md, Coding Guidelines
Aufgabenbezogen	Feature-Specs, API-Docs, Modul-Doku	Spec fuer aktuelles Ticket
Temporaer	Recherche, Debugging-Kontext	Logfiles, Stacktraces

Tools zur Kontextverwaltung

- `.github/copilot-instructions.md` — Copilot-spezifisch, immer geladen
- **AGENTS.md** — Tool-agnostisch, wachsender Standard
- **VS Code Workspaces** — mehrere Repos als Kontext einbinden
- **MCP-Server** — dynamischer Kontext (DB-Schema, Jira-Tickets, Git-Historie)
- **Context7** — aktuelle Library-Dokumentation on demand

Quellen & Weiterlesen — Kontextmanagement

- [Martin Fowler — Context Engineering for Coding Agents](#) — Grundlagen und Patterns fuer kontextbewusstes Agenten-Design
- [Anthropic — Effective context engineering for AI agents](#) — Empfehlungen direkt vom Hersteller
- [GitHub Blog — How to write a great agents.md](#) — Lessons from over 2,500 repositories
- [AGENTS.md Specification — ASDLC](#) — Offizielle Spec und Best Practices
- [JetBrains Research — Smarter Context Management for LLM-Powered Agents](#) — Effizientes Kontext-Management in der Praxis

Dokumentation als Steuerungsinstrument

Warum Dokumentation wieder wichtig wird: Sie ist fuer den Agenten.

- Fehlende Doku = Agent erfindet oder raet
- Gute Doku = besserer, vorhersagbarer Output
- Dokumentation wird zum **Steuerungsinstrument**, nicht zum Papiertiger

Welche Formate fuer Agenten?

Format	Wofuer	Agent-Verstaendnis
Markdown	Alles	★★★ Exzellent
UML (Mermaid)	Architektur, Ablaeufe	★★☆ Gut
ADRs	Designentscheidungen	★★★ Exzellent
Code-Beispiele	Konventionen zeigen	★★★ Exzellent
Coding Guidelines	Standards durchsetzen	★★★ Exzellent

NFRs festhalten — auch fuer den Agenten

- **Performance:** Latenz-Budgets, SLAs, Skalierung
- **Security:** OWASP-Anforderungen, Compliance-Regeln
- **Verfuegbarkeit:** SLAs, Fallback-Strategien
- **Format:** ADRs oder NFR-Sektion in Architektur-Doku
- **Wo:** Schicht 1 (immer aktiv) — Agent muss NFRs bei jeder Aenderung kennen

3 Schichten der Dokumentation

1. **Immer aktiv:** Konventionen, Architektur, Plattform-Grundlagen
2. **Aufgabenbezogen:** Feature-Specs, API-Docs, Modul-Dokumentation
3. **Temporaer:** Recherche-Ergebnisse, Debugging-Kontext

Dokumentation als lebendes Projekt — bei jedem Feature mitaktualisieren.

Quellen & Weiterlesen — Dokumentation als Steuerungsinstrument

- [Mintlify — What to Include in AGENTS.md](#) — Welche Doku-Inhalte Agenten wirklich brauchen
- [AI Hero — A Complete Guide To AGENTS.md](#) — Praxisleitfaden fuer agentengerechte Dokumentation
- [Harness — The Agent-Native Repo: Why AGENTS.MD is the New Standard](#) — ADRs, Guidelines und Doku-Schichten im Ueberblick
- [GitHub Blog — How to write a great agents.md](#) — Lessons from over 2,500 repositories
- [Elastic — What is Context Engineering? Architecting Reliable AI](#) — Doku als Kontext-Fundament fuer zuverlaessige Agenten

Brainstorming & Planning mit dem Agenten

Anforderung → Brainstorming → Spec → Plan → Execution

1. **Brainstorming:** Anforderung verstehen, Optionen erkunden
2. **Spec:** Technische Spezifikation schreiben
3. **Plan:** Aufgaben in 5-20min Tasks zerlegen (TDD)
4. **Execution:** Task fuer Task umsetzen mit Entwickler-Checkpoints

Wann volle Methodik, wann reduziert?

Aufgabe	Methodik
Neues Modul / Feature	Brainstorming → Spec → Plan → TDD
Groesseres Refactoring	Plan → TDD
API-Aenderung	Spec → Plan → TDD
Kleine UI-Aenderung	Direkter Prompt mit Test
Bugfix (lokal)	Failing Test → Fix
Config-Aenderung	Direkter Prompt

Ausblick: Subagenten & Multi-Agent

- **Subagenten:** Spezialisierte Agenten fuer Teilaufgaben
 - Recherche-Agent, Test-Agent, Review-Agent
- **Koordination:** Hauptagent delegiert und integriert
- **Praxis heute:** Claude Code Subagents, GitHub Copilot Agent Mode

Noch frueh — aber die Richtung ist klar.

Quellen & Weiterlesen — Brainstorming und Planning

- [Claude Code: Best Practices](#) — Offizielle Empfehlungen von Anthropic: Workflow, Planning und Execution mit Claude Code
- [Dev.to — The AI Coding Workflow That Actually Works: Separate Planning from Execution](#) — Warum Planung und Umsetzung getrennt werden sollten
- [superpowers — writing-plans Skill \(SKILL.md\)](#) — Praxisvorlage fuer strukturierte Planung mit dem Agenten
- [InfoQ — From Prompts to Production: a Playbook for Agentic Development](#) — Vollstaendiges Playbook vom Prompt bis zum Deployment
- [Thoughtworks — Preparing your team for the agentic software development life cycle](#) — Team-Vorbereitung auf den Agentic SDLC

Modelle und ihre Staerken

Aufgabe	Empfohlenes Modell	Warum
Planning	Claude Opus/Sonnet 4.6	Beste Instruktionsbefolgung
Coding	Codex 5.3, Claude Sonnet 4.6	Schnell + praezise
Review	Claude 4.6, Gemini 3.1	Regelanalyse, grosser Kontext
Recherche	Gemini 3.1	Grosses Kontextfenster

Kosten im Griff behalten

- **Premium-Requests:** Entstehen bei Nutzung leistungsstaerkerer Modelle
- **Einsparen durch:**
 - Richtiges Modell fuer die richtige Aufgabe
 - Guten Kontext (weniger Iterationen = weniger Requests)
 - Standard-Modelle fuer einfache Aufgaben
- **Investition:** Bessere Methodik → weniger Nacharbeit → weniger Kosten

Quellen & Weiterlesen — Modelle und Kosten

- [GitHub Copilot Plans & Pricing](#) — Offizielle Uebersicht aller Plaene mit Premium-Request-Kontingenten und Preisen
- [GitHub Docs: Supported AI Models](#) — Welche Modelle in Copilot verfuegbar sind und ihre Premium-Request-Multiplier
- [GitHub Docs: Model Comparison](#) — Detaillierter Vergleich der Copilot-Modelle nach Staerken und Anwendungsfall
- [Microsoft Tech Community: Choosing the Right Model in GitHub Copilot](#) — Praxisleitfaden zur Modellwahl fuer Entwickler
- [NxCode: Claude Sonnet 4.6 vs Opus 4.6](#) — Detailvergleich: Benchmarks, Kosten, Empfehlung fuer welche Aufgaben